Sovereign Enterprise AI

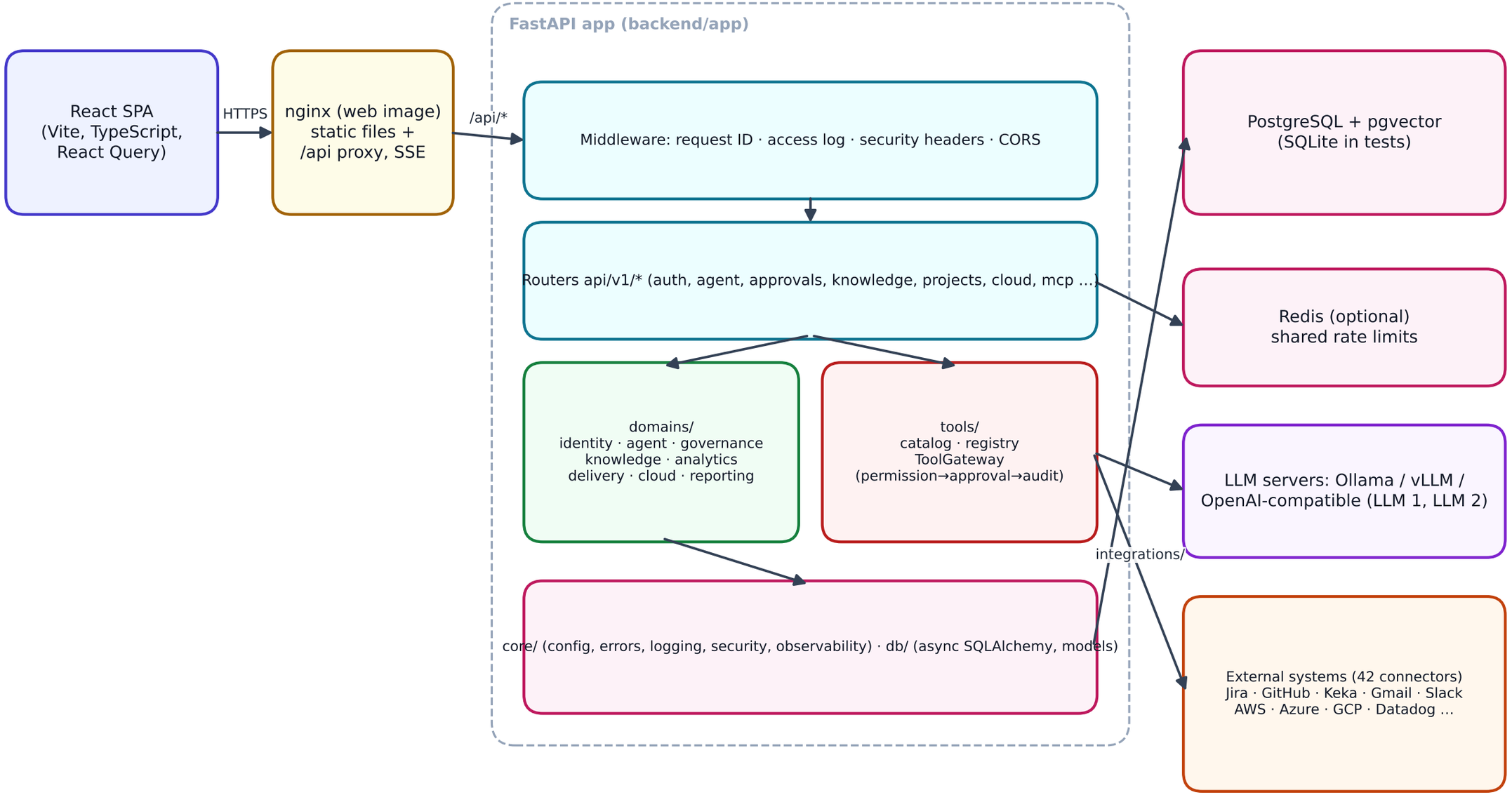
System Architecture Diagrams

Architecture · sequences · deployment · integrations · AWS · Azure · GCP · RAG · MCP · agent

1. architecture
2. layers
3. request lifecycle
4. agent graph
5. tool gateway
6. approval sequence
7. rag ingestion
8. hybrid retrieval
9. platform indexing
10. aws flow
11. azure flow
12. gcp flow
13. docker compose
14. kubernetes helm
15. ci pipeline
16. mcp code mode
17. auth flow
18. audit chain
19. delay analysis
20. analytics
21. data model
22. scaling

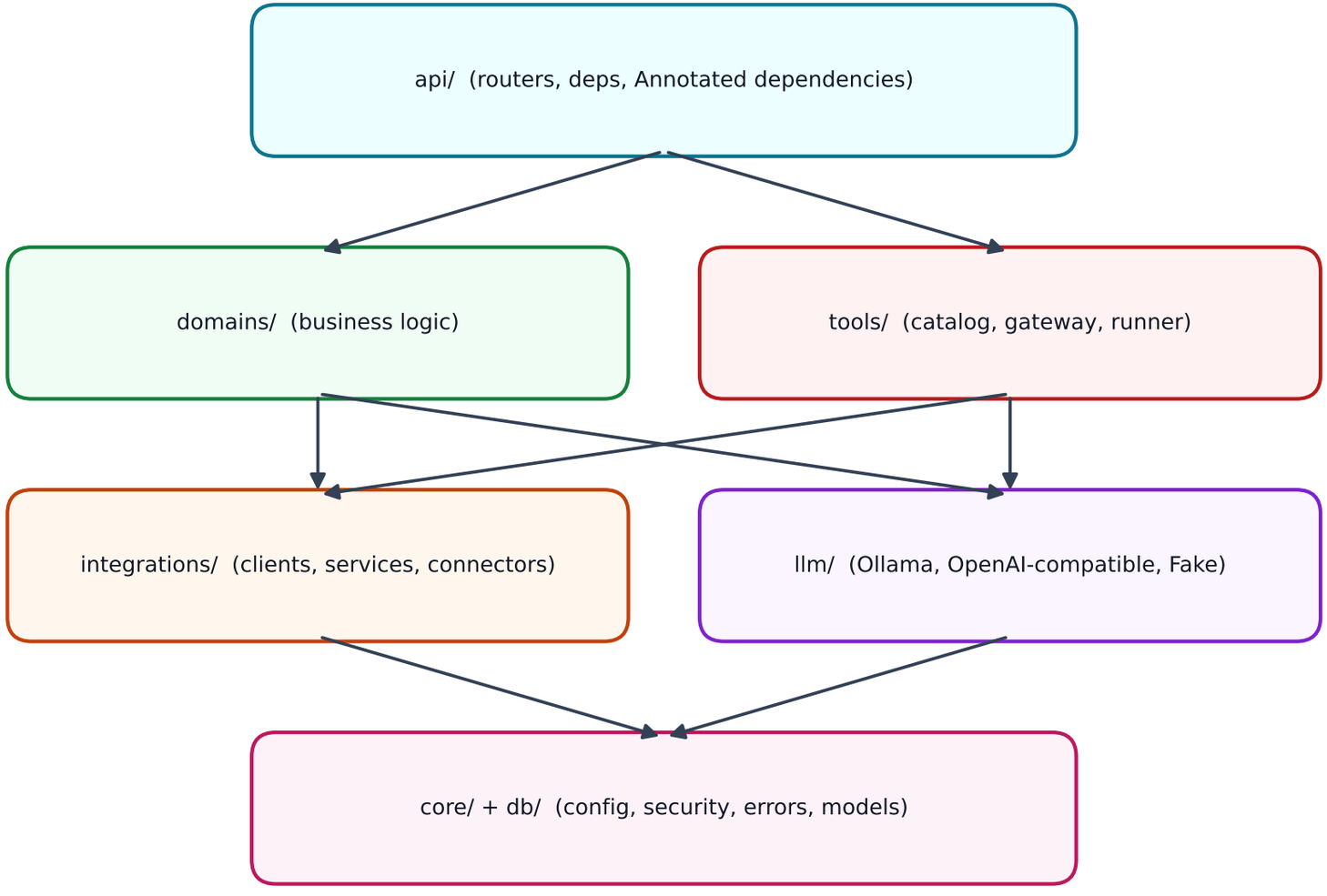
1. High-level architecture

Modular monolith: one FastAPI process, domain modules, governed tool layer



2. Dependency direction (layers)

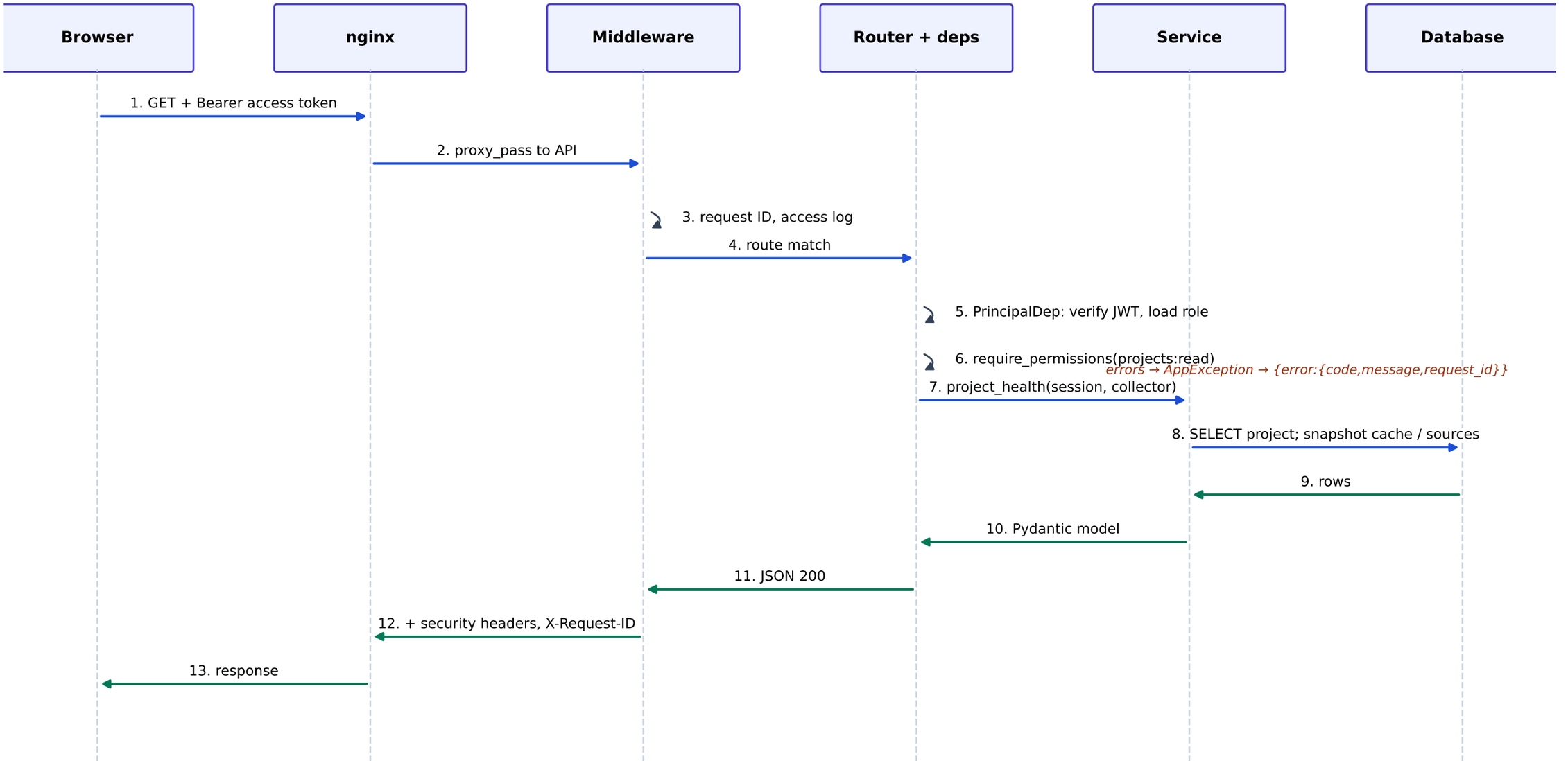
Arrows point to what a layer may import; integrations never import domains



db/models/__init__.py imports every domain model (Alembic, tests and imports rely on it).

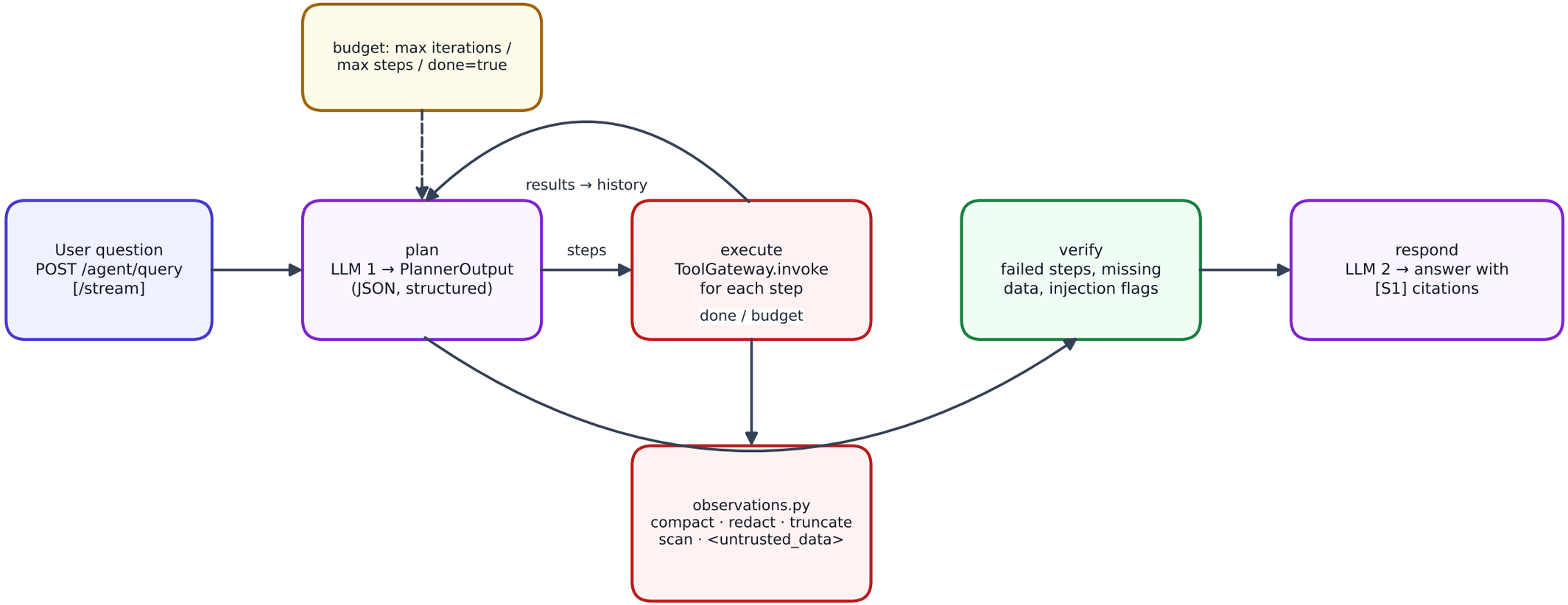
3. Request lifecycle (GET /api/v1/projects/PHX/health)

From the browser to the database and back



4. Agent graph (LangGraph StateGraph)

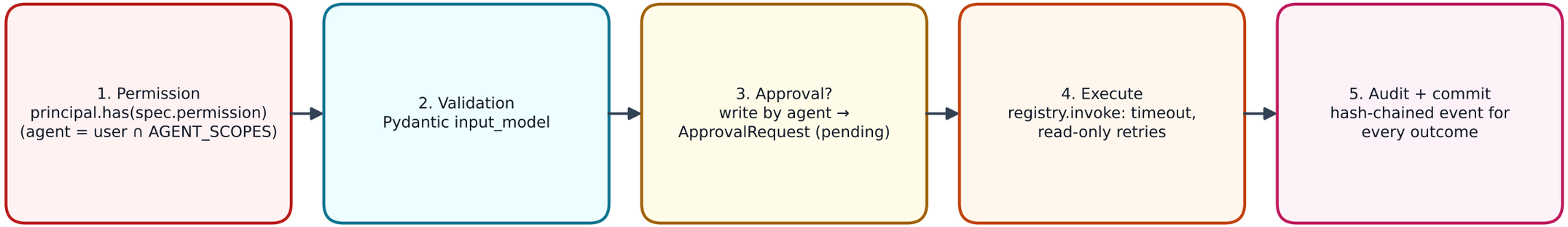
domains/agent/graph.py: plan → execute (loop) → verify → respond, streamed as SSE



SSE events: run_started → step* → phase → done | error. The stream task uses its own DB session and finishes even if the client disconnects.

5. ToolGateway: every tool call, every caller

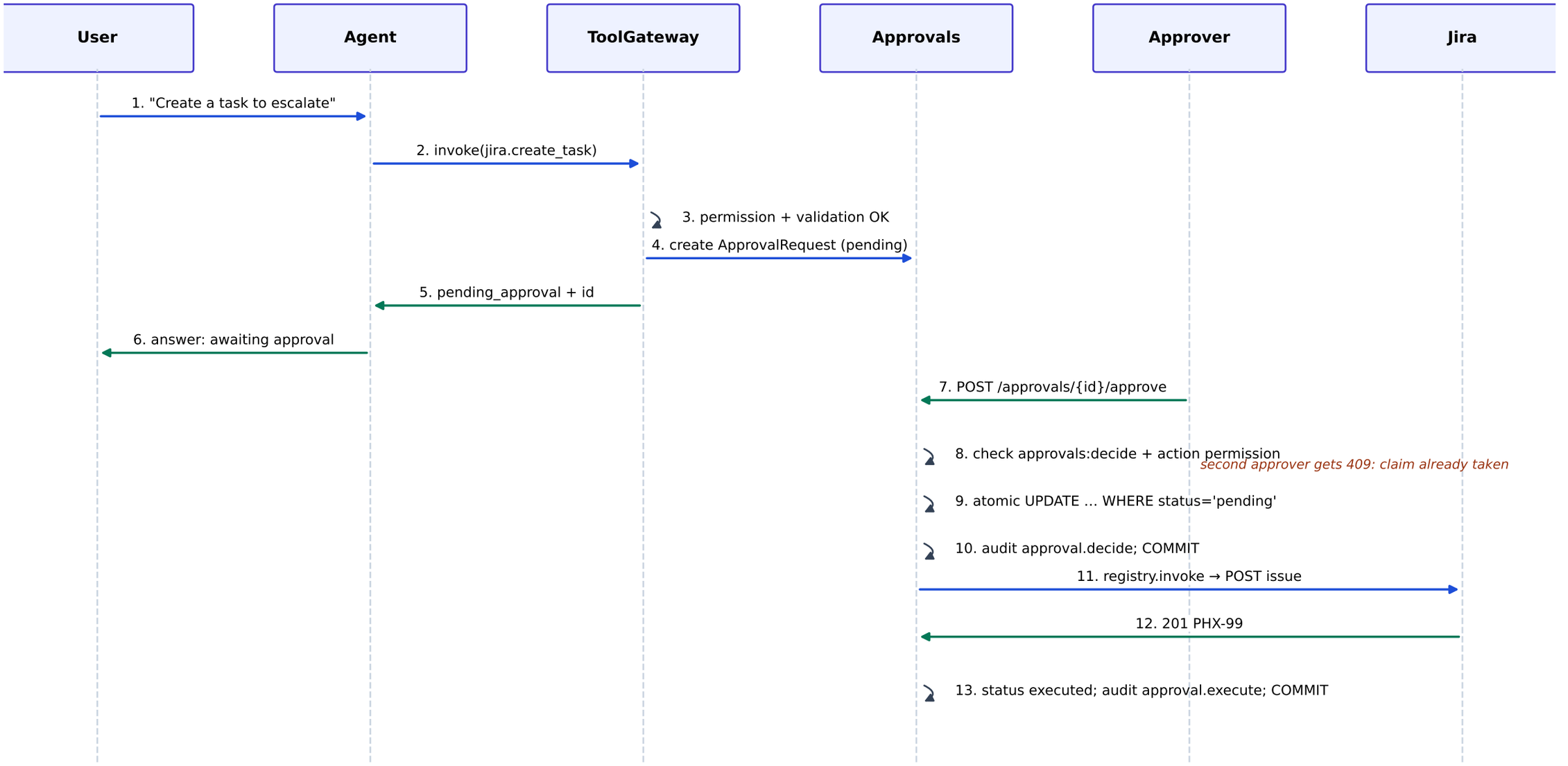
tools/gateway.py — agent, MCP code mode, briefing, backlog submit all go through invoke()



Denied → status denied + audit. Invalid → error + audit. Pending → approval id returned to the planner.
principal_var (core/context.py) is set during execution so per-user tools know who is asking.

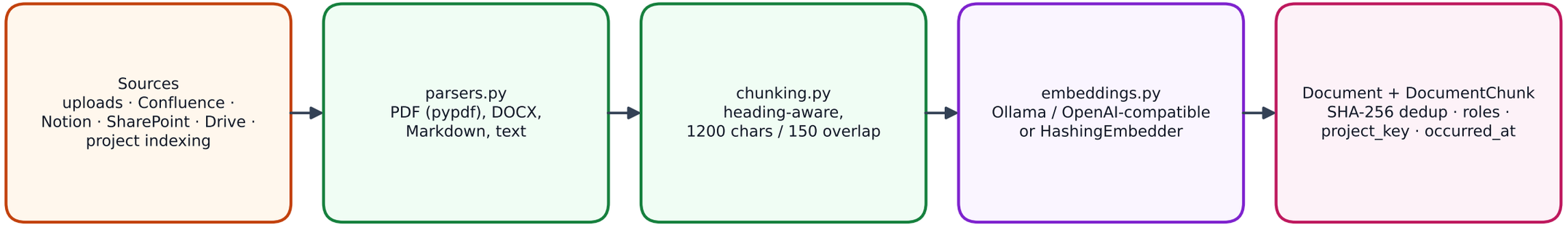
6. Sensitive write: from request to audit record

jira.create_task proposed by the agent, approved by a person



7. RAG ingestion

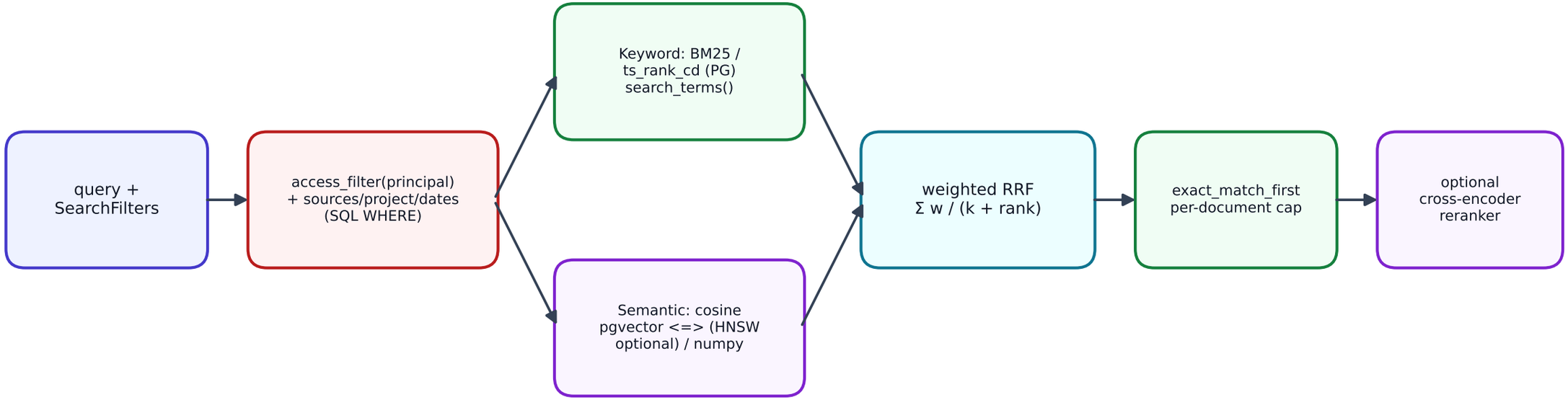
domains/knowledge: upload / sync / index → chunks with embeddings and keyword terms



External embedding endpoint → RedactingEmbedder removes secrets/PII first. Unknown source ACL → admin-only (fail closed).
Incremental: unchanged checksum skipped; changed re-embedded; vanished removed only after a complete listing.

8. Hybrid retrieval with weighted RRF

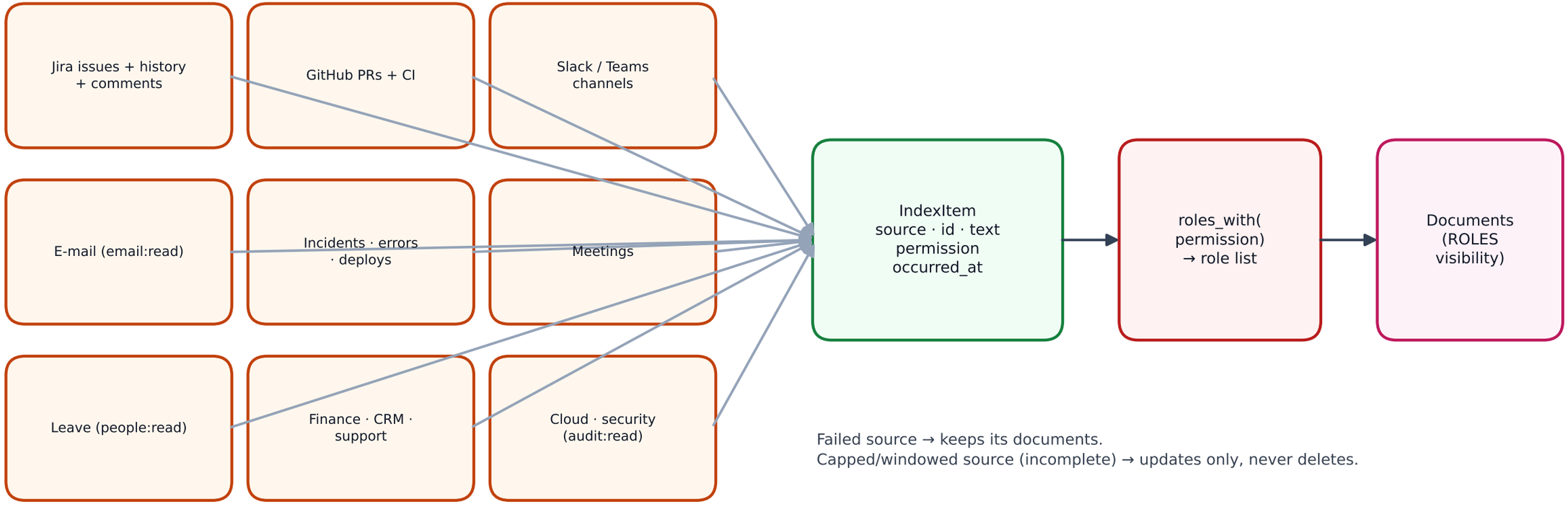
KnowledgeService.search: access control and filters are SQL WHERE clauses applied before ranking



fusion_params: real embedder → k=60, weights 1:1. Hashing embedder → k=5, semantic weight 0.1 (tuned with the retrieval eval).

9. Indexing every connected platform per project

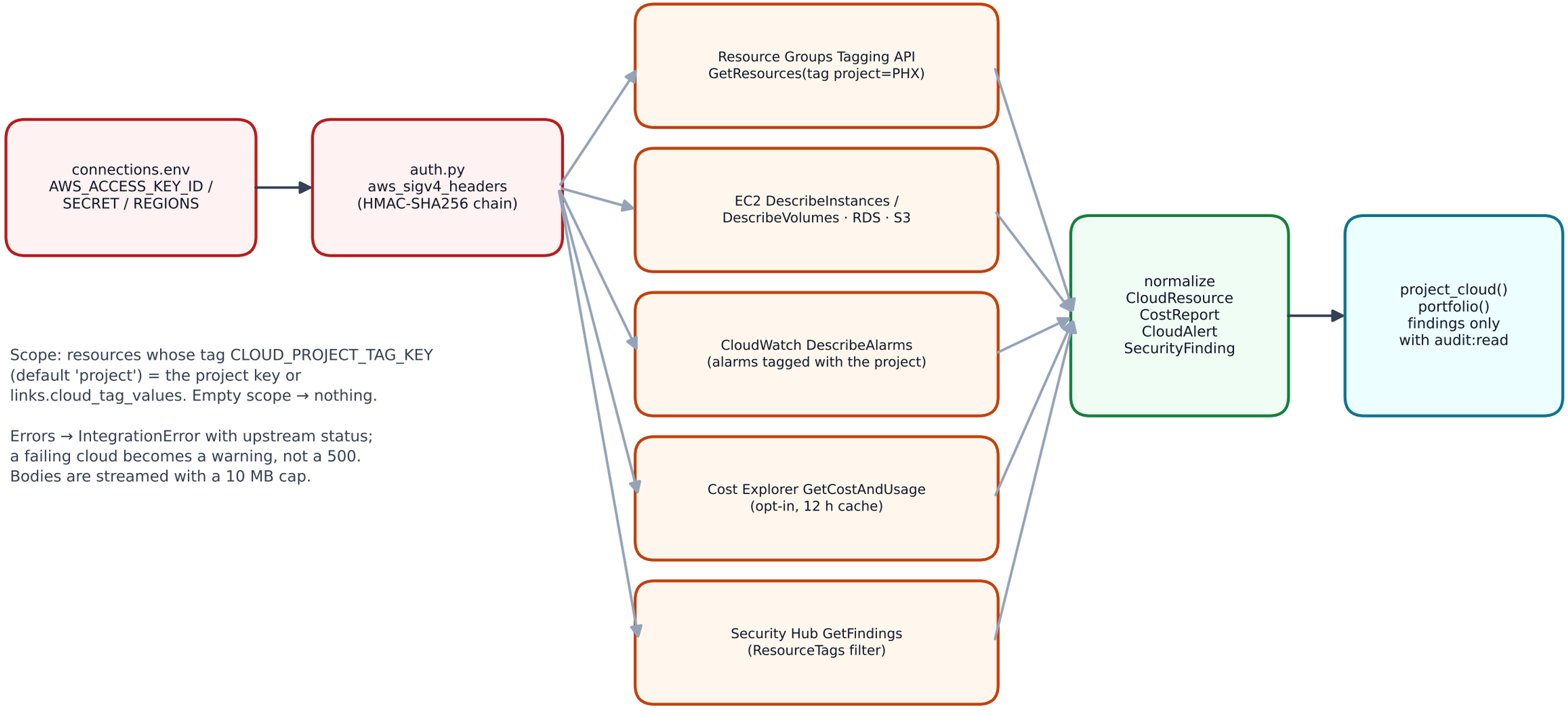
domains/knowledge/indexing.py — one document per record, with the permission needed to read it



Failed source → keeps its documents.
Capped/windowed source (incomplete) → updates only, never deletes.

10. How AWS data is fetched (read-only monitoring)

integrations/connectors/cloud.py AwsCloudService → domains/cloud/service.py → /api/v1/projects/{key}/cloud

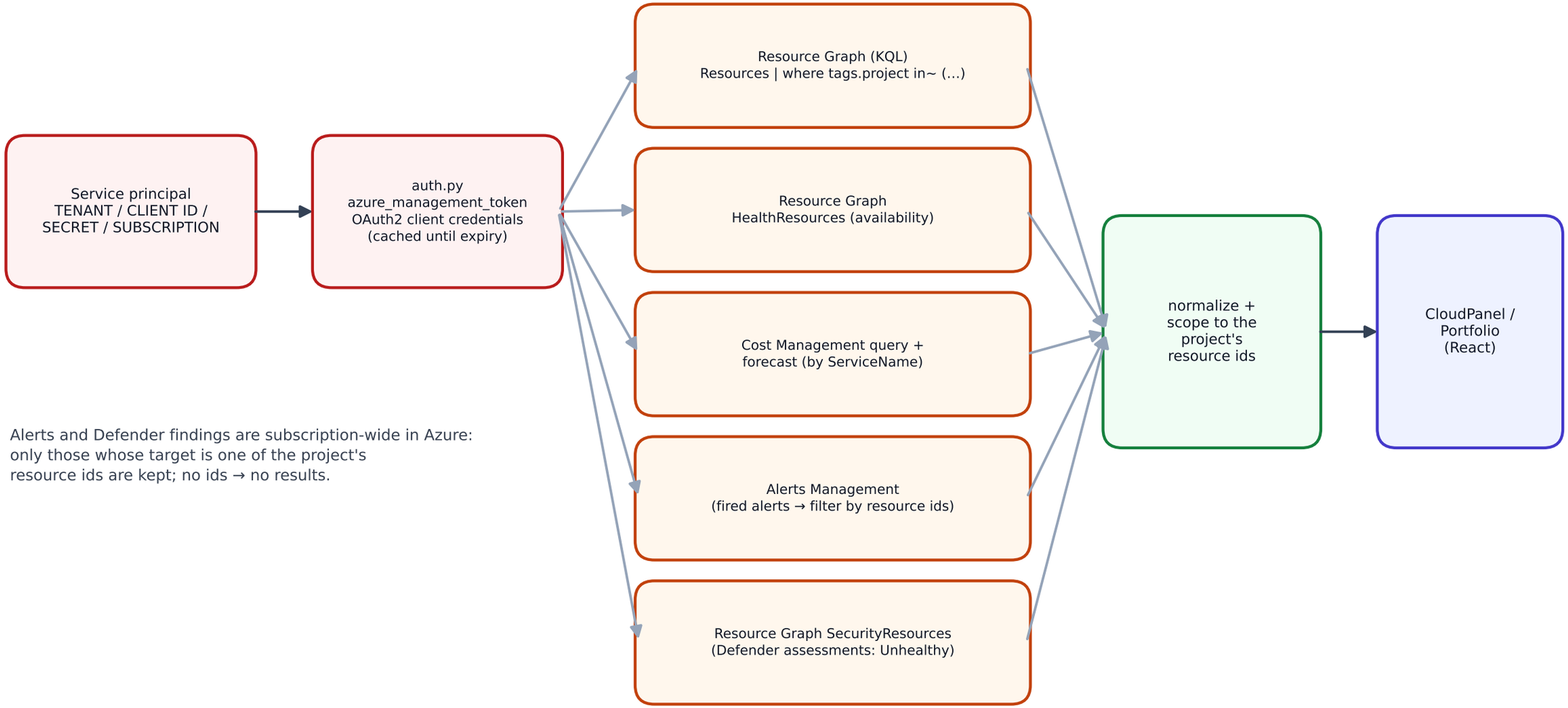


Scope: resources whose tag CLOUD_PROJECT_TAG_KEY (default 'project') = the project key or links.cloud_tag_values. Empty scope → nothing.

Errors → IntegrationError with upstream status; a failing cloud becomes a warning, not a 500. Bodies are streamed with a 10 MB cap.

11. How Azure data is fetched

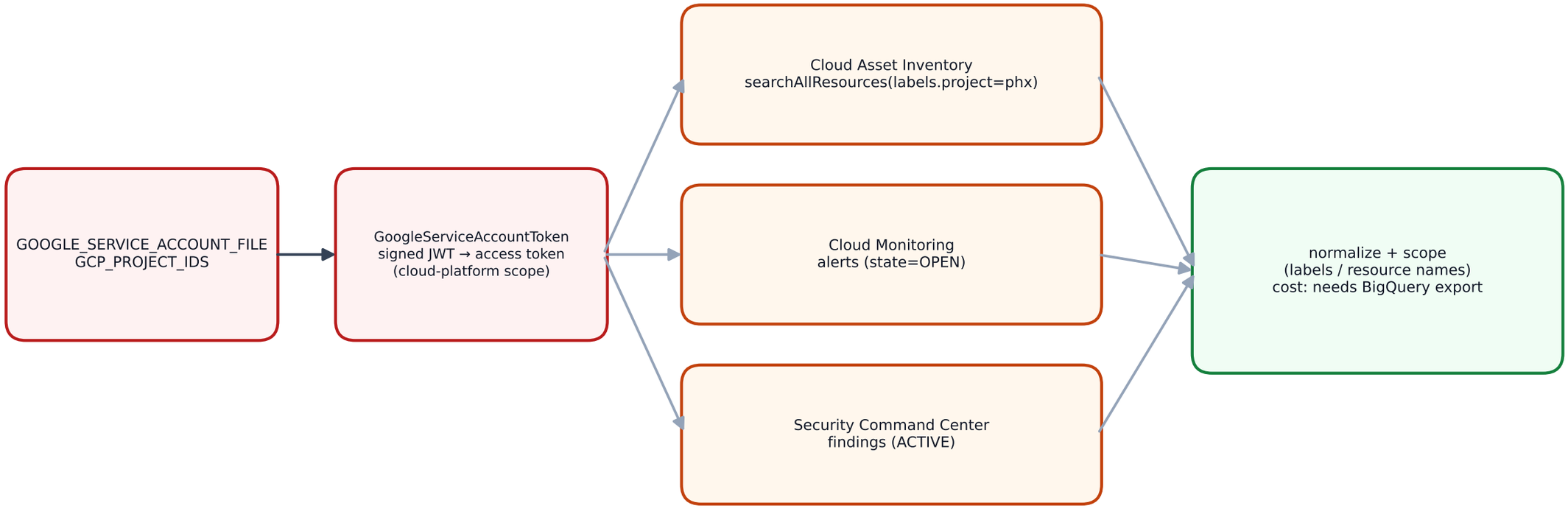
AzureCloudService: Entra ID client-credentials token → Azure Resource Manager APIs



Alerts and Defender findings are subscription-wide in Azure: only those whose target is one of the project's resource ids are kept; no ids → no results.

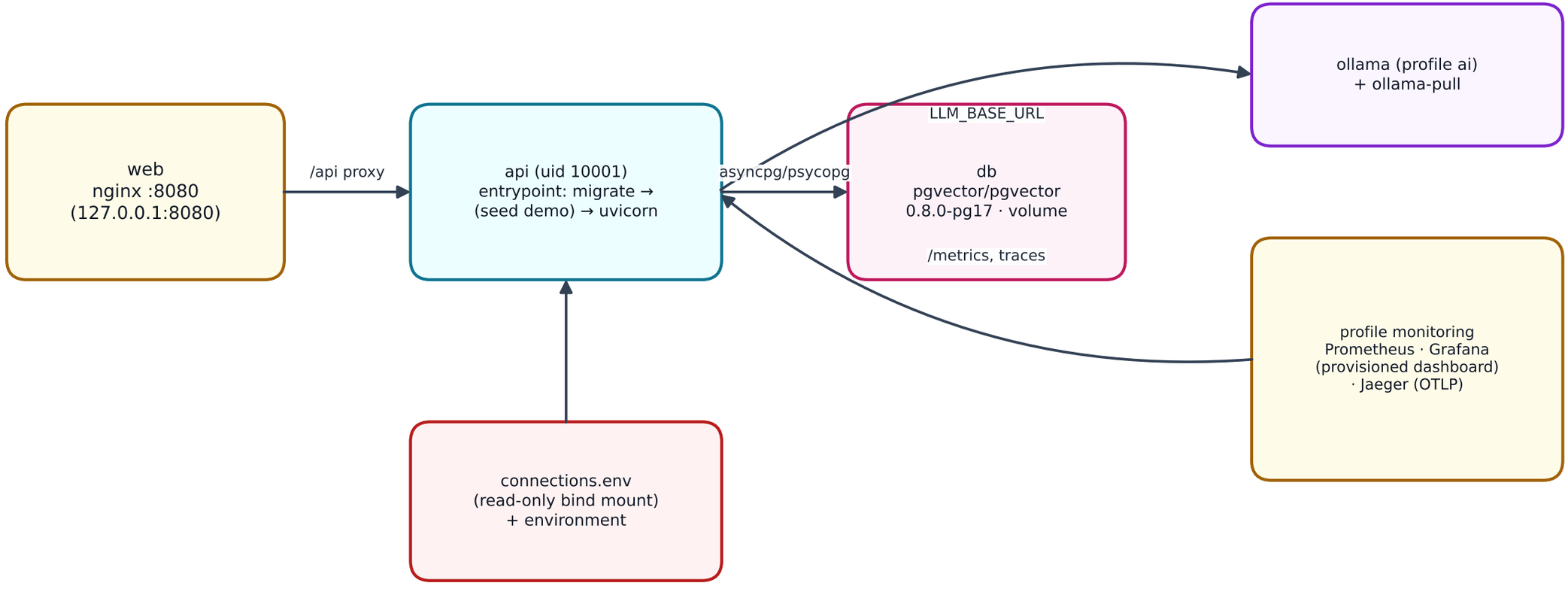
12. How Google Cloud data is fetched

GcpCloudService: service-account JWT → OAuth token → Google APIs (no cost API)



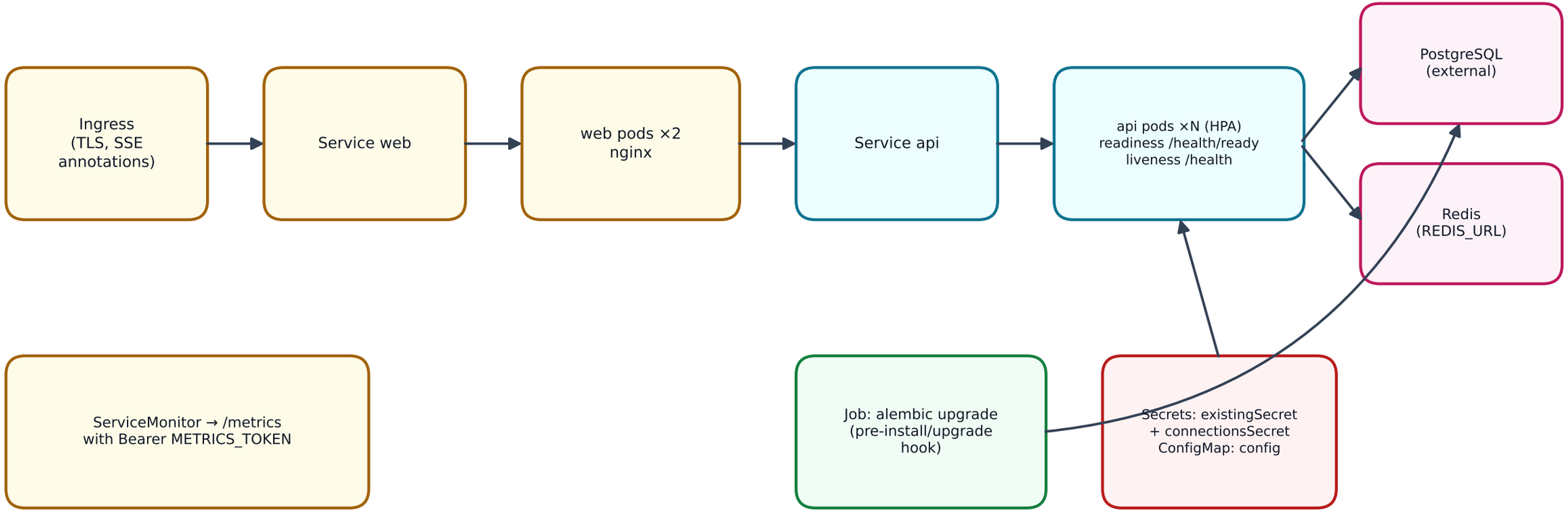
13. Docker Compose deployment

docker-compose.yml — default: db + api + web; profiles: ai (Ollama), monitoring (Prometheus, Grafana, Jaeger)



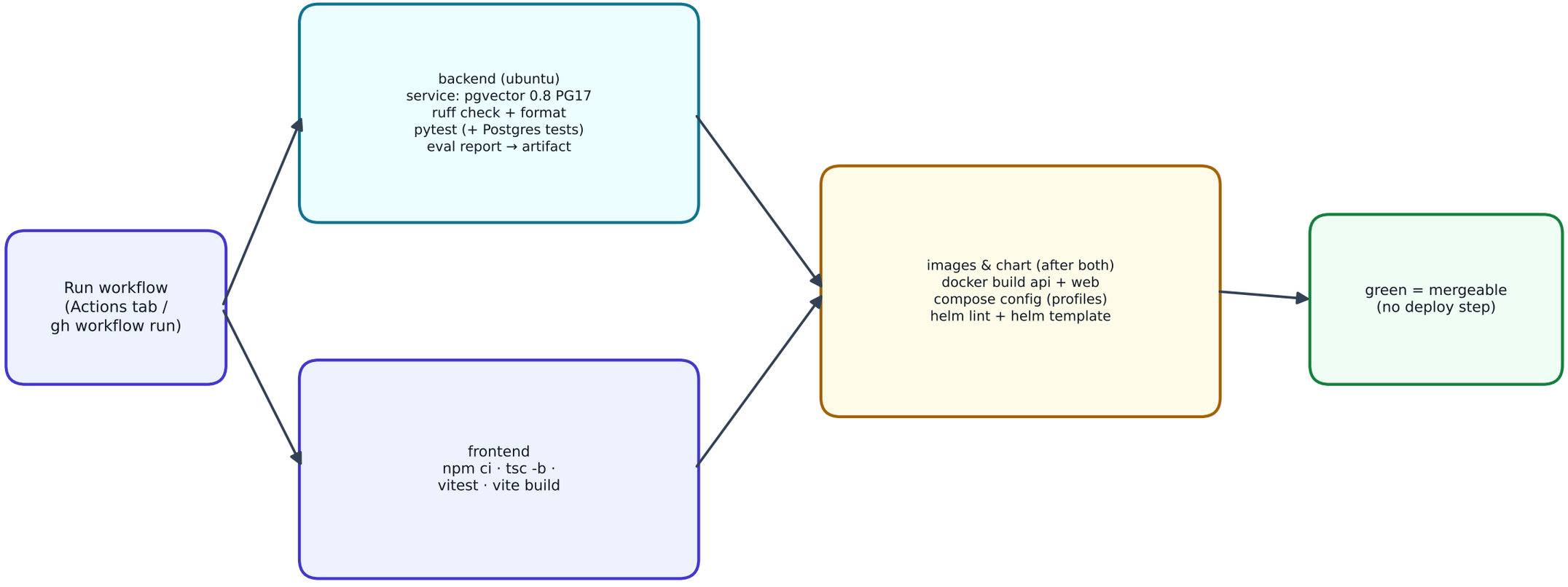
14. Kubernetes with the Helm chart

deploy/helm/sovereign — external PostgreSQL, secrets from existingSecret, migrations as a hook Job



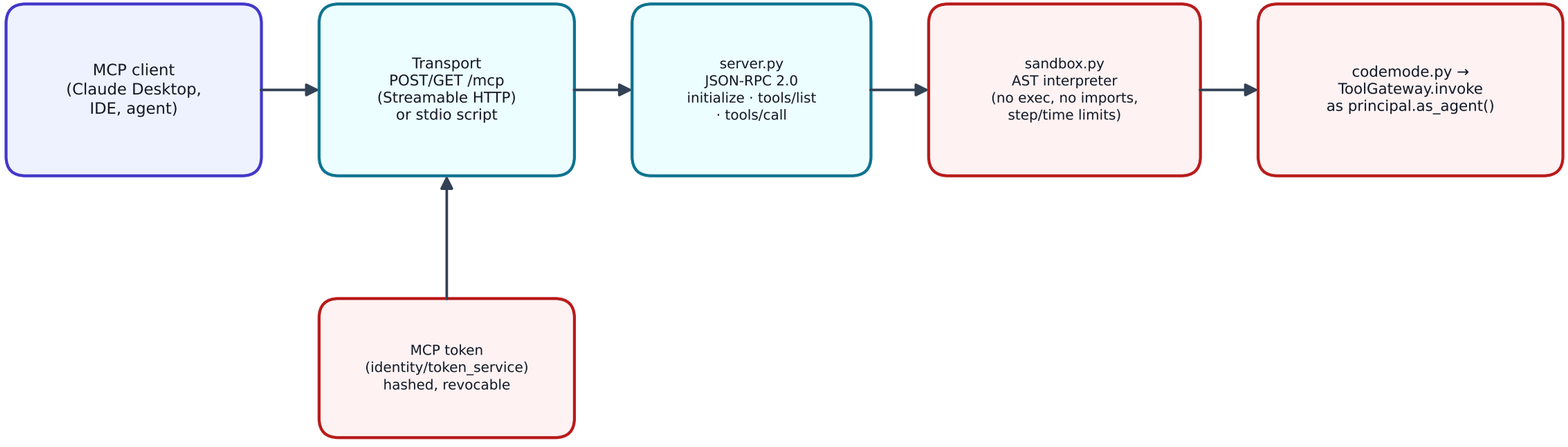
15. GitHub Actions CI

.github/workflows/ci.yml — manual (workflow_dispatch) so it never spends Actions minutes unasked



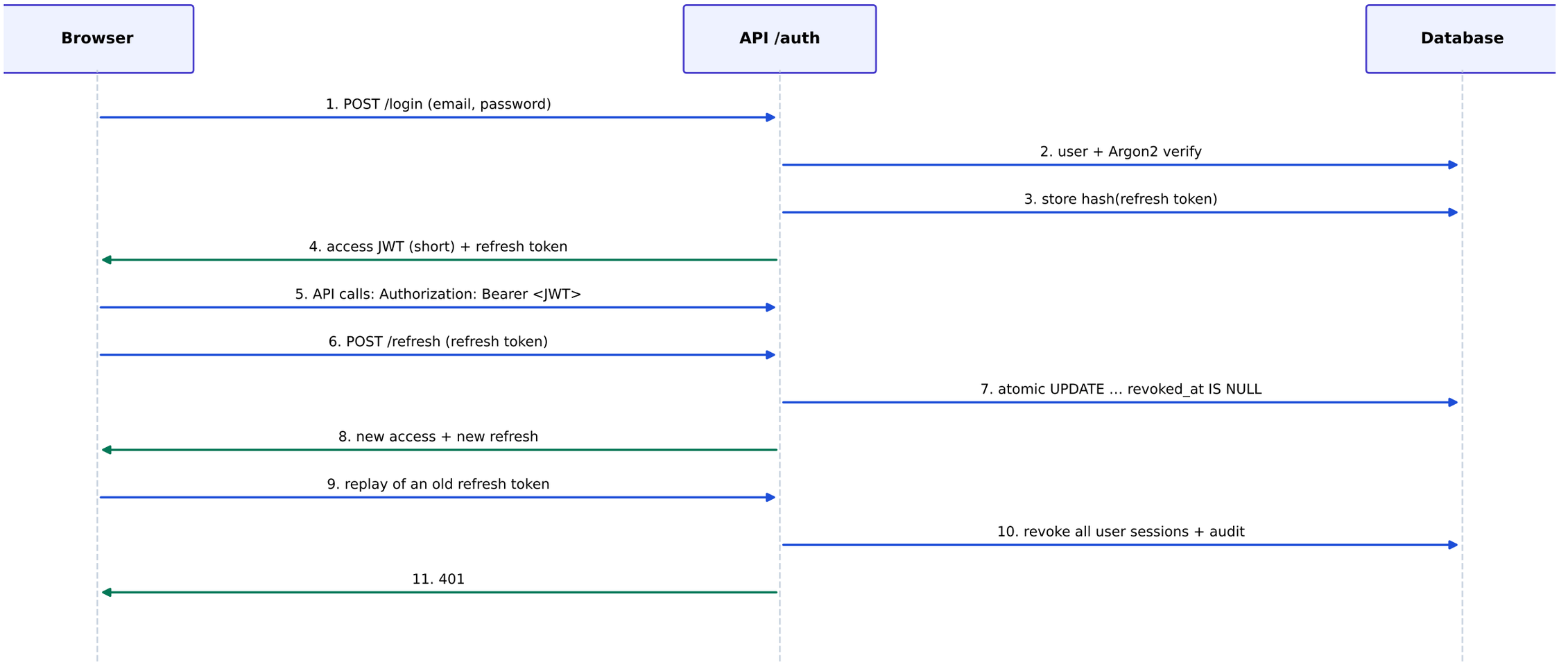
16. MCP server in code mode

domains/mcp — the client writes a small script; a sandboxed AST interpreter runs it; every call goes through ToolGateway



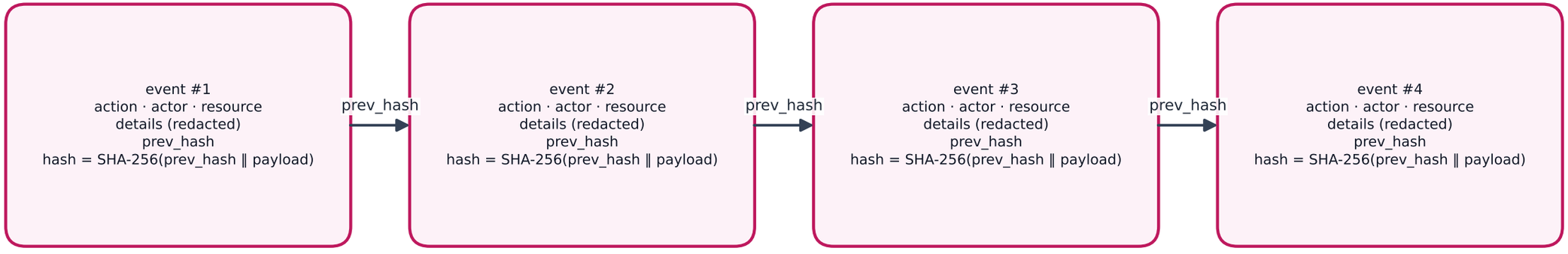
17. Authentication: access tokens and refresh rotation

core/security + domains/identity/auth_service.py



18. Hash-chained audit log

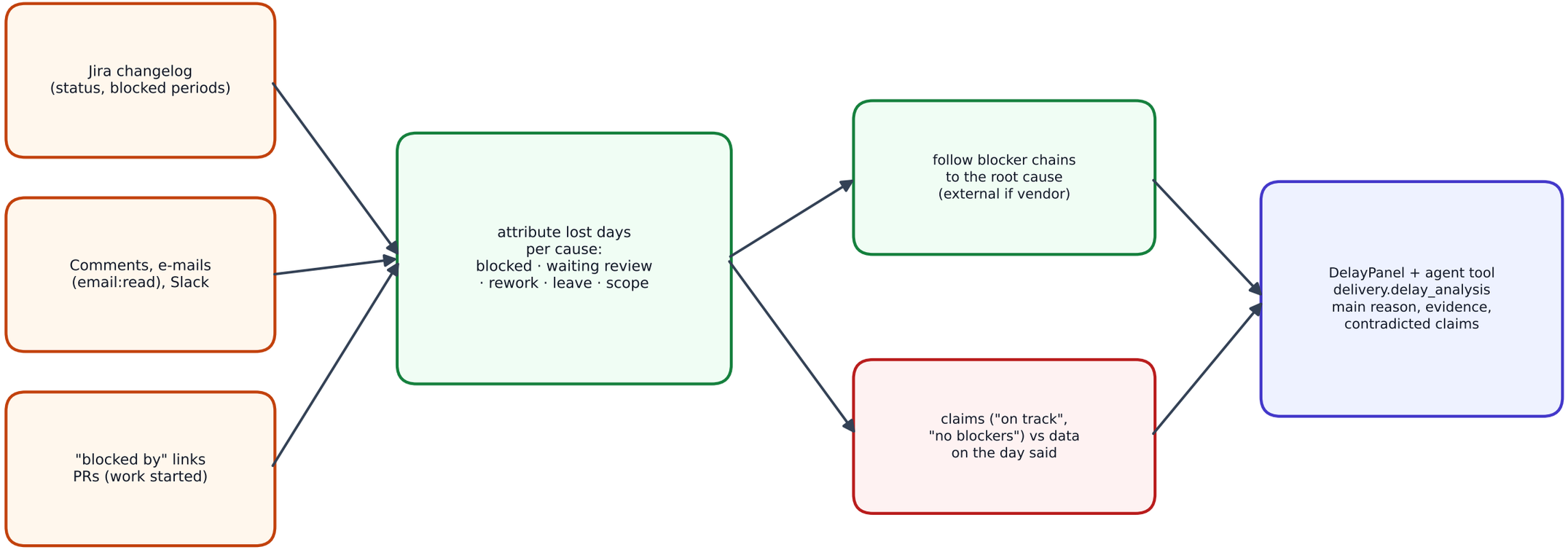
domains/governance/audit/service.py — written in the caller's transaction after locking audit_chain_head



Changing any past event changes its hash, so every later prev_hash stops matching: GET /audit/verify reports the first broken link.
Events are append-only (never updated or deleted). The head row is locked (SELECT … FOR UPDATE) so concurrent writers serialise.

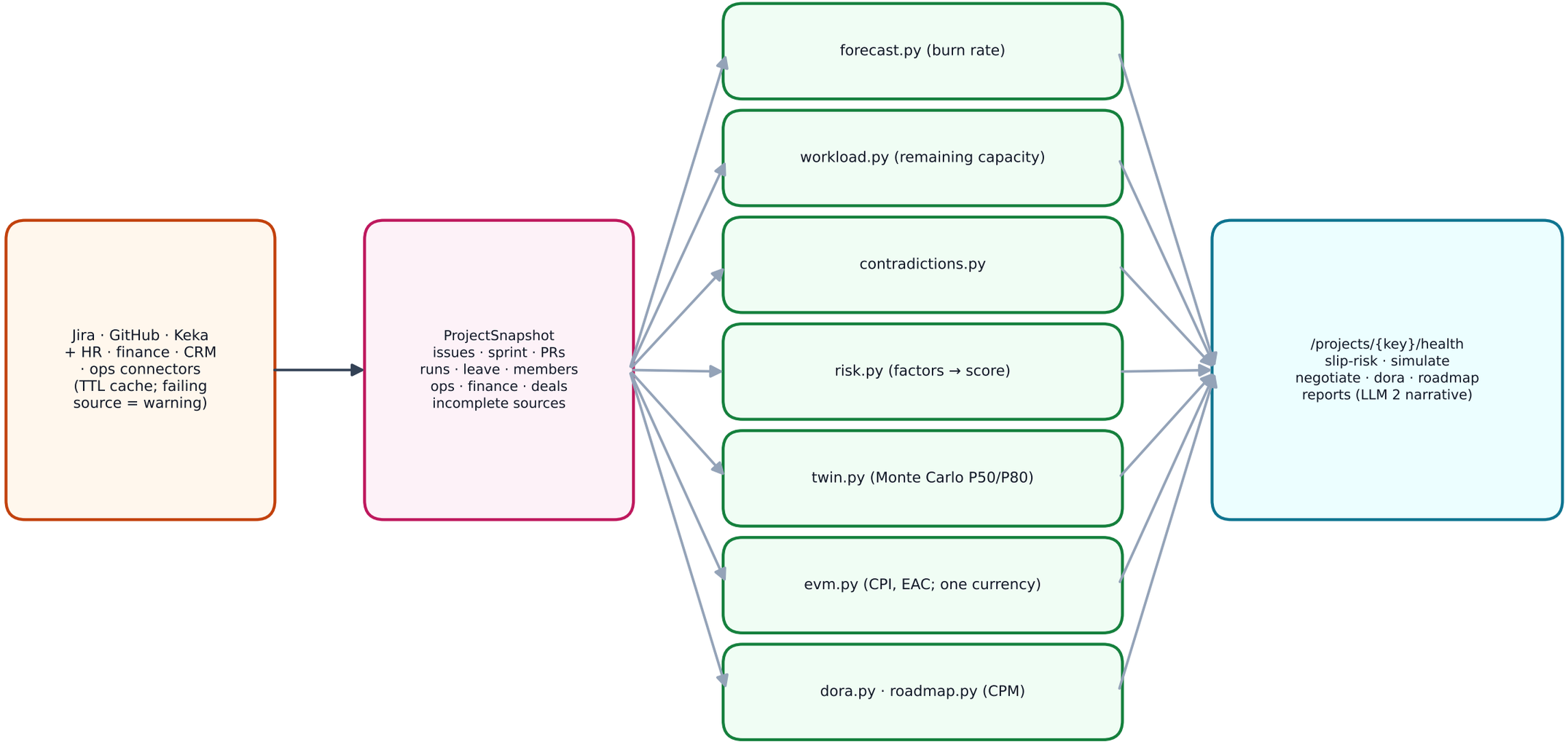
19. "Why is it late?" — delay analysis

domains/delivery/delay.py — person-days lost per cause, root causes and claim checks



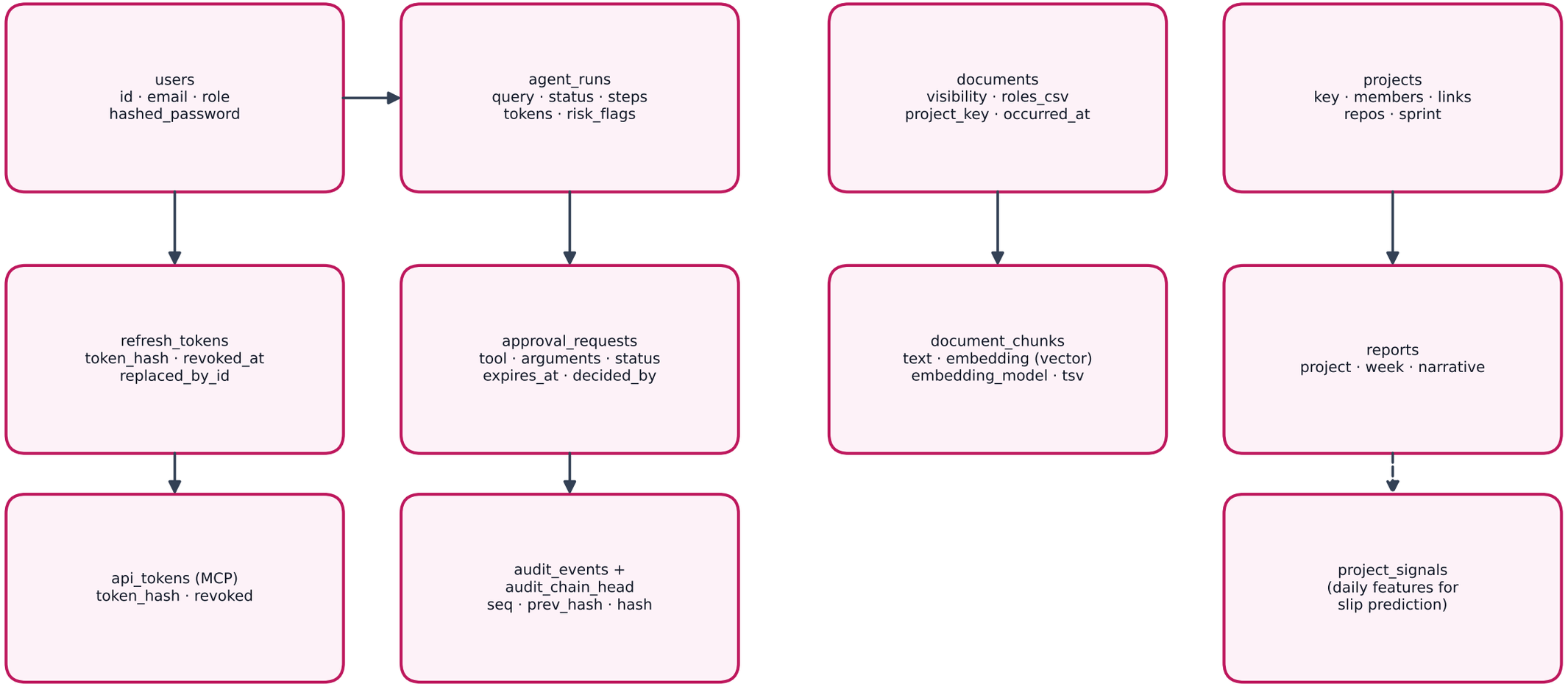
20. Analytics: one snapshot, many pure functions

domains/analytics/collector.py → snapshot.py facts → forecast / workload / risk / twin / EVM



21. Core data model

SQLAlchemy 2 models (registered in db/models/__init__.py), migrations 0001–0008



22. Scaling out

What is shared when there are several API processes — and what is not built yet

